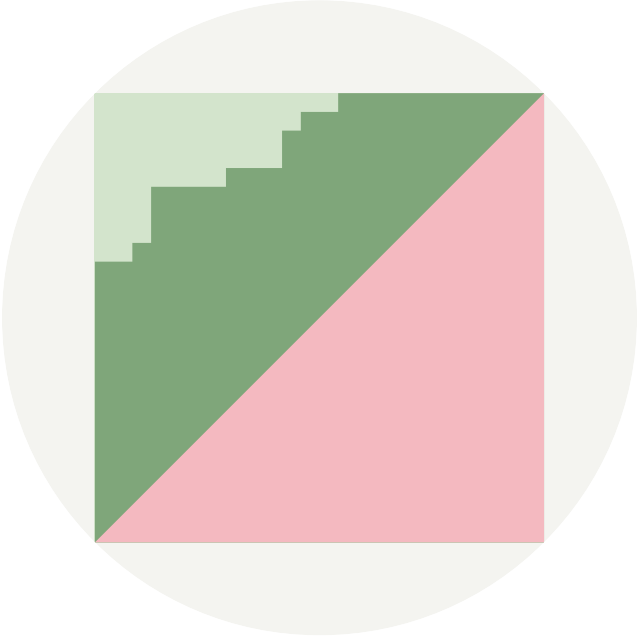
Данные и метрики

Теория



Тур

- ШЭ 2026, 9–11 классы: 120 минут, 72 балла, шесть задач — четыре с ответом и две программы.
- Ответ проверяется после тура, засчитывается последняя посылка: ошибку в формате никто не покажет. Формат — дробь, округление, порядок чисел — перечитать перед отправкой.
- Программа проверяется сразу на тестах, засчитывается лучшая посылка. Сдаётся текстом `.py`, не ноутбуком; ввод — стандартный, ничего лишнего в выводе: ни подсказок, ни отладки.
- Файл к задаче на данные дают в XLSX, ODS или CSV. Брать CSV: для XLSX pandas нужен `openpyxl`, для ODS — `odfpy`, а ставить пакеты на туре негде.

CSV

Текст: строка файла — строка таблицы, поля через разделитель, первая строка обычно заголовок. Поле с разделителем, кавычкой или переводом строки берут в двойные кавычки, кавычку внутри удваивают:

```
id,fio,komment
7,"Петров, Иван","сказал ""да"""
```

Русский Excel сохраняет CSV по-своему: разделитель — точка с запятой, десятичная — запятая, кодировка — cp1251 или UTF-8 с меткой BOM в начале файла. Поэтому **сначала файл смотрят глазами**, как текст:

```
with open("f.csv", encoding="utf-8") as f:
    print(repr(f.read(300)))
```

`repr` показывает разделитель, кавычки, в начале — это BOM — и концы строк.

Def (read_csv). Нужные параметры:

- `sep=";"`, `decimal=","` — разделитель и десятичная;
- `encoding="utf-8-sig"` — съедает BOM; `"cp1251"` — старый русский Excel;
- `na_values=["-", "н/д"]` — что ещё считать пропуском, кроме пустого поля и `NA`;
- `dtype={"id": str}` — не терять ведущие нули;
- `header=None, names=[...]` — файл без заголовка;
- `skipinitialspace=True` — пробелы после разделителя;
- `parse_dates=["time"]` — сразу разобрать даты.

Def (после чтения). Сразу:

```
df.shape, df.head(), df.dtypes
df.isna().sum()
df.columns.tolist()
```

Столбец чисел с типом `object` или `str` — в нём есть не число. Имя столбца `"id"` или `" price"` — BOM или пробел, и `df["id"]` упадёт с `KeyError`.

Чистка

- Строки: `s.str.strip()`, `s.str.lower()`. Чем заполнен столбец, видно по `s.value_counts()`: значение с пробелом на конце — отдельное значение.
- Числа из мусора: `pd.to_numeric(s, errors="coerce")` — всё, что не число, станет `NaN`. Сначала `str.strip()`.
- Особое значение, которое условие велит считать пропуском, — `s.replace(v, np.nan)` или `na_values` при чтении.
- Повторы: `df.drop_duplicates()` — по всей строке, `subset=[...]` — по части столбцов.
- Фильтр: `df[(df.x >= a) & (df.y < b)]`, каждое условие в скобках; `s.between(a, b)` включает оба конца.
- Даты: `pd.to_datetime(s)`, дальше `s.dt.date`, `s.dt.hour`, `s.dt.dayofweek`; разность дат — `Timedelta`.

Группы и соединения

- `df.groupby("k")["v"].agg(["mean", "median", "count"])`. `count` не считает пропуски, `size` — считает. Строки с пропуском в ключе `groupby` по умолчанию выбрасывает.
- `.transform(f)` возвращает значение группы на каждой строке — столбец той же длины, что таблица.
- `a.merge(b, on="id", how="left")`. Ключи одного типа: `42` и `"0042"` не совпадут. `indicator=True` добавляет столбец `_merge`: `left_only`, `both`, `right_only`. Есть ли пара — `a.id.isin(b.id)`.
- Сортировка с разбором ничьих:

```
df.sort_values(["cnt", "w"],
               ascending=[False, True])
```

  `idxmax` — первый максимум; `mode()[0]` — наименьшая из мод; порядок равных в `value_counts` не гарантирован.
- Ответ файлом: `df.to_csv("answer.csv", index=False)` — без индекса.

Числа

- `round` в питоне банковский: `round(2.5) == 2`, `round(0.125, 2) == 0.12`. Школьное округление:

```
from decimal import Decimal, ROUND_HALF_UP
Decimal(str(x)).quantize(Decimal("0.001"),
                         rounding=ROUND_HALF_UP)
```

- Ответ дробью — считать в `fractions.Fraction` с самого начала: `Fraction(3, 5)`, печать даёт `3/5` уже несократимой.
- `np.std` делит на n, `pd.Series.std` — на $n-1$. Писать `ddof` явно.
- Квартили: `quantile` по умолчанию интерполирует. Если в условии квартиль — медиана половины, считать по условию.
- Доля верных — среднее булевых: `(pred == y).mean()`.

Метрики

Классификация: TP, FP, FN, TN — верно и ложно положительные, ложно и верно отрицательные; $P = TP + FN$, $N = FP + TN$.

$$\text{accuracy} = \frac{TP + TN}{P + N}, \qquad \text{precision} = \frac{TP}{TP + FP},$$

$$\text{recall} = \frac{TP}{P}, \qquad \text{specificity} = \frac{TN}{N},$$

$$F_1 = \frac{2\,TP}{2\,TP + FP + FN}, \qquad \text{FPR} = 1 - \text{specificity}.$$

Recall ещё называют TPR. ROC-AUC — доля пар «положительный, отрицательный», где у положительного оценка выше, ничья — половина; площадь под ROC-кривой.

Регрессия, $e_i = y_i - \hat{y}_i$:

$$\text{MAE} = \tfrac{1}{n}\sum |e_i|, \quad \text{MSE} = \tfrac{1}{n}\sum e_i^2,$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \qquad R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}.$$

MAPE — среднее $|e_i/y_i|$ в процентах. Лучшая константа под MSE — среднее, под MAE — медиана.

Модели: неоднородность узла дерева — Джини $1 - \sum p_k^2$ или энтропия $-\sum p_k \log_2 p_k$; близость векторов — косинус $\frac{u \cdot v}{|u|\,|v|}$; у метода соседей признаки разного масштаба сначала приводят к одному.

Программы

Быстрый ввод и вывод:

```
import sys
data = sys.stdin.buffer.read().split()
n = int(data[0])
a = list(map(int, data[1:n + 1]))
...
sys.stdout.write("\n".join(map(str, ans)) + "\n")
```

`input()` в цикле на $2 \cdot 10^5$ строк ощутимо медленнее. Текст со строками целиком — `sys.stdin.read()`.

- Питон делает порядка 10^7 простых операций в секунду. При $n = 2 \cdot 10^5$ квадрат не пройдёт, нужно $n \log n$ или n.
- Что под рукой: `sorted` с `key`, `bisect`, `collections.Counter`, `defaultdict`, `deque` — очередь с обоих концов за $O(1)$, `heapq`, `itertools`.
- Рекурсия глубже тысячи падает: циклом, или `sys.setrecursionlimit`.
- Перед сдачей — крайние случаи: $n = 1$, все равны, отрицательные, самые большие числа.